Economic Factors and Quality of Life A Global Data Analysis

Final project report
Data Collection and Preparation - SIS

Executive summary

This project combines World Bank API indicators with Wikipedia demographic data to study how economic and demographic factors relate to life expectancy in 2023. The World Bank source covers 217 economies and the scrape covers 244 locations. After 36 explicit country-name mappings, the final integrated dataset contains **216 countries and territories** with no duplicate rows.

The main result is a strong, non-linear income-health association: raw GDP per capita has Pearson r = **0.62** with life expectancy, while log GDP has Pearson r = **0.85** and Spearman rho = **0.86**. The richest GDP quartile averages 81.6 years versus 65.2 in the poorest quartile.

Research question and pipeline

The question is how GDP per capita, unemployment, population, density and area are related to life expectancy. The workflow is World Bank API collection, Wikipedia scraping, cleaning, country harmonization, merge diagnostics, EDA, correlation analysis, visualization and an optional regression extension.

Project part	Main work	Output
Person 1	World Bank API, economic cleaning and correlations	world_bank_economic_2023.csv
Person 2	Wikipedia scraping, harmonization and integration	wikipedia_countries.csv and final_dataset.csv
Person 3	EDA, visualization and conclusions	figures and final notebook

World Bank API collection

The API was queried in JSON format for 2023 using GDP per capita, life expectancy, unemployment and population. Country metadata removed regional aggregates, ISO3 codes supported integration, nested labels were cleaned, and get_indicator reused the same procedure for each indicator.

Four indicators for 2023 are downloaded from the World Bank API. Regional/income aggregates (e.g. "Arab World") are removed using the API's country metadata, so only real countries and territories remain.

In [3]:

```
import pandas as pd
import requests
import json
import numpy as np

url = "https://api.worldbank.org/v2/country/all/indicator/NY.GDP.PCAP.CD"

r = requests.get(
    url,
    params={
        "format": "json",
        "date": 2023,
        "per_page": 300
    }
)

r.raise_for_status()

data_gdp = json.loads(r.content.decode("utf-8-sig"))

meta_gdp = data_gdp[0]
records_gdp = data_gdp[1]

df_gdp = pd.DataFrame(records_gdp)

df_gdp.head()
```

Out[3]:

	indicator	country	countryiso3code	date	value	unit	obs_status	decimal
0	{'id': 'NY.GDP.PCAP.CD', 'value': 'GDP per cap...	{'id': 'ZH', 'value': 'Africa Eastern and Sout...	AFE	2023	1571.132704			1
1	{'id': 'NY.GDP.PCAP.CD', 'value': 'GDP per cap...	{'id': 'ZI', 'value': 'Africa Western and Cent...	AFW	2023	1846.246811			1
2	{'id': 'NY.GDP.PCAP.CD', 'value': 'GDP per cap...	{'id': '1A', 'value': 'Arab World'}	ARB	2023	7504.356377			1

Original Jupyter cell 3: World Bank API request

In [13]:

```python
def get_indicator(code, column_name):
    url = f"https://api.worldbank.org/v2/country/all/indicator/{code}"

    r = requests.get(url, params={"format": "json","date": 2023,"per_page": 300})

    r.raise_for_status()

    data = json.loads(r.content.decode("utf-8-sig"))
    records = data[1]

    df = pd.DataFrame(records)


    df = df[df["countryiso3code"].isin(valid_iso3)].copy()

    df["Country"] = df["country"].apply(
        lambda x: x["value"] if isinstance(x, dict) else x
    )

    df = df.rename(columns={
        "countryiso3code": "iso3",
        "date": "year",
        "value": column_name
    })

    df[column_name] = pd.to_numeric(
        df[column_name],
        errors="coerce"
    )

    return df[
        ["iso3", "Country", "year", column_name]
    ]
```

Original Jupyter cell 13: reusable get_indicator function

Wikipedia scraping

Requests and BeautifulSoup extracted the first wikitable. Non-breaking spaces and separators were removed, values were converted to numeric types, Antarctica was removed because its population was missing, and parenthetical labels were stripped. The scrape contains 244 locations.

In [27]:

```python
import requests

url = "https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population_density"
headers = {"User-Agent": "SIS-project"}
response = requests.get(url, headers=headers)
print(response.status_code)
```

```
200
```

In [28]:

```python
from bs4 import BeautifulSoup

soup = BeautifulSoup(response.text, "html.parser")
tables = soup.find_all("table", class_="wikitable")
print(len(tables))
```

```
1
```

In [29]:

```python
table = tables[0]
rows = table.find_all("tr")
print(len(rows))
print(rows[0].text)
print(rows[1].text)
```

```
248
Location
Density
Population
Land area
Ref.

/km2
/mi2
km2
mi2
```

Original Jupyter cells 27-29: requests and BeautifulSoup scraping

Integration and harmonization

The first outer merge produced 180 matches, 64 Wikipedia-only rows and 37 World Bank-only rows. Thirty-six explicit mappings harmonized names. The second merge produced 216 matches, 28 Wikipedia-only dependencies and one World Bank-only aggregate. The final inner merge contains 216 rows and 11 columns, with no duplicate rows.

In [48]:
```
final = pd.merge(df, wb, on="Country", how="inner")
print(final.shape)
print(final.columns.tolist())
```

```
(216, 11)
['Country', 'Density', 'Population', 'Area', 'iso3', 'year', 'gdp_per_capita', 'life_expectancy', 'unemployment', 'population', 'log_gdp']
```

In [49]:
```
final = final.rename(columns={
    "Population": "population_wiki",
    "population": "population_wb",
    "Density": "density_km2",
    "Area": "area_km2",
})
print(final.columns.tolist())
```

```
['Country', 'density_km2', 'population_wiki', 'area_km2', 'iso3', 'year', 'gdp_per_capita', 'life_expectancy', 'unemployment', 'population_wb', 'log_gdp']
```

In [50]:
```
print(final.isna().sum())
print(final.duplicated().sum())
print(final["Country"].duplicated().sum())
```

```
Country             0
density_km2         0
population_wiki     0
area_km2            0
iso3                0
year                0
gdp_per_capita     13
life_expectancy     0
unemployment       33
population_wb       0
log_gdp            13
dtype: int64
0
0
```

Original Jupyter cells 48-50: final merge and duplicate checks

Density agrees with population divided by area within 5% for 99.5% of rows. The median Wikipedia/World Bank population ratio is 1.012, and 95.8% are within 10%.

Final integrated dataset preview

Country	iso3	gdp_per_capita	life_expectancy	unemployment	density_km2
Macao SAR, China	MAC	$67,216	83.18	2.21	21,826
Monaco	MCO	$256,800	86.37	-	19,428
Singapore	SGP	$86,383	83.10	3.44	8,290
Hong Kong SAR, China	HKG	$50,525	85.25	2.95	7,062
Gibraltar	GIB	-	83.55	-	5,800
Bahrain	BHR	$29,290	81.28	1.05	2,034
Maldives	MDV	$12,588	81.04	4.02	1,759
Malta	MLT	$40,933	83.36	3.48	1,686

Figure 4. Final integrated dataset preview.

EDA and data quality

The final dataset has 216 rows and 11 columns. There are 13 missing GDP values and 33 missing unemployment values. No imputation was performed. GDP is strongly right-skewed, while life expectancy is comparatively symmetric.

In [54]:

```
missing = pd.DataFrame({"missing": df.isna().sum(), "percent": (df.isna().mean() * 100).round(1)})
missing = missing[missing["missing"] > 0]
print(missing)
print("\nCountries without GDP per capita:", df.loc[df["gdp_per_capita"].isna(), "Country"].tolist())
```

```
                missing  percent
gdp_per_capita       13      6.0
unemployment         33     15.3
log_gdp              13      6.0

Countries without GDP per capita: ['Gibraltar', 'St. Martin (French part)', 'Guam', 'British Virgin Islands', 'Virgin Islands (U.S.)', 'American Samoa', "Korea, Dem. People's Rep.", 'Syrian Arab Republic', 'Northern Mariana Islands', 'Cuba', 'Yemen, Rep.', 'Eritrea', 'South Sudan']
```

Descriptive statistics

In [55]:

```
num_cols = ["gdp_per_capita", "life_expectancy", "unemployment", "population_wb", "density_km2", "area_km2"]
desc = df[num_cols].describe().T
desc["skew"] = df[num_cols].skew()
desc.round(2)
```

Out[55]:

	count	mean	std	min	25%	50%	75%	max	skew
gdp_per_capita	203.0	22037.95	3.343954e+04	249.80	2925.79	7827.30	29777.07	2.567999e+05	3.40
life_expectancy	216.0	73.76	7.120000e+00	54.46	68.45	74.52	79.00	8.637000e+01	-0.41
unemployment	183.0	6.78	5.540000e+00	0.13	3.15	5.18	8.50	3.461000e+01	2.07
population_wb	216.0	37219363.16	1.424833e+08	9816.00	844378.50	6723398.00	26478633.50	1.438070e+09	8.77
density_km2	216.0	471.71	2.137610e+03	0.03	40.00	97.50	246.00	2.182600e+04	8.39
area_km2	216.0	615823.71	1.784233e+06	2.00	10202.50	92943.00	470625.00	1.712519e+07	5.95

Original Jupyter cells 54-55: missing values and descriptive statistics

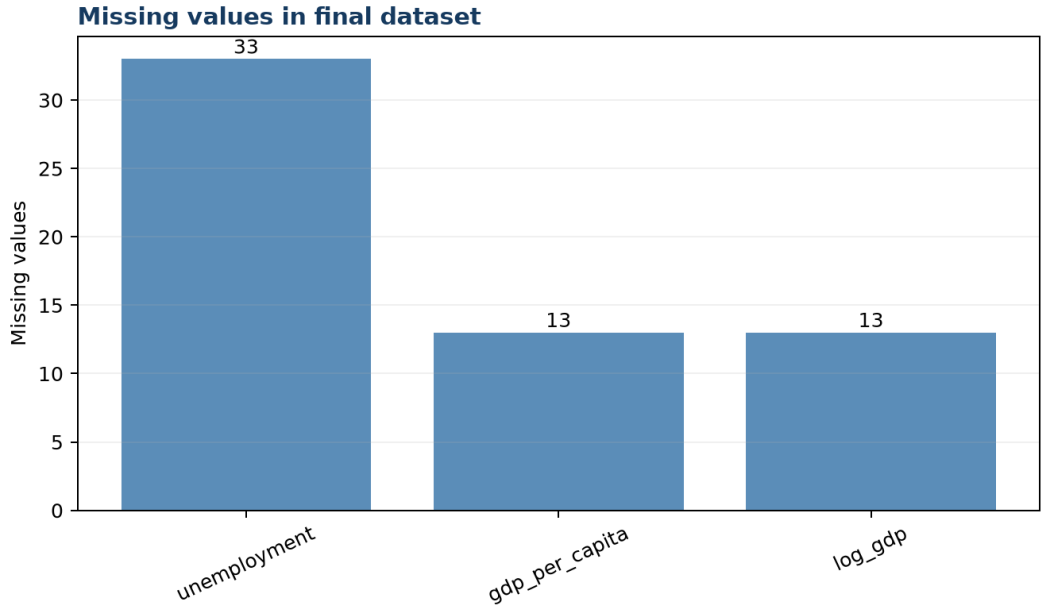


Figure 6. Missing values.

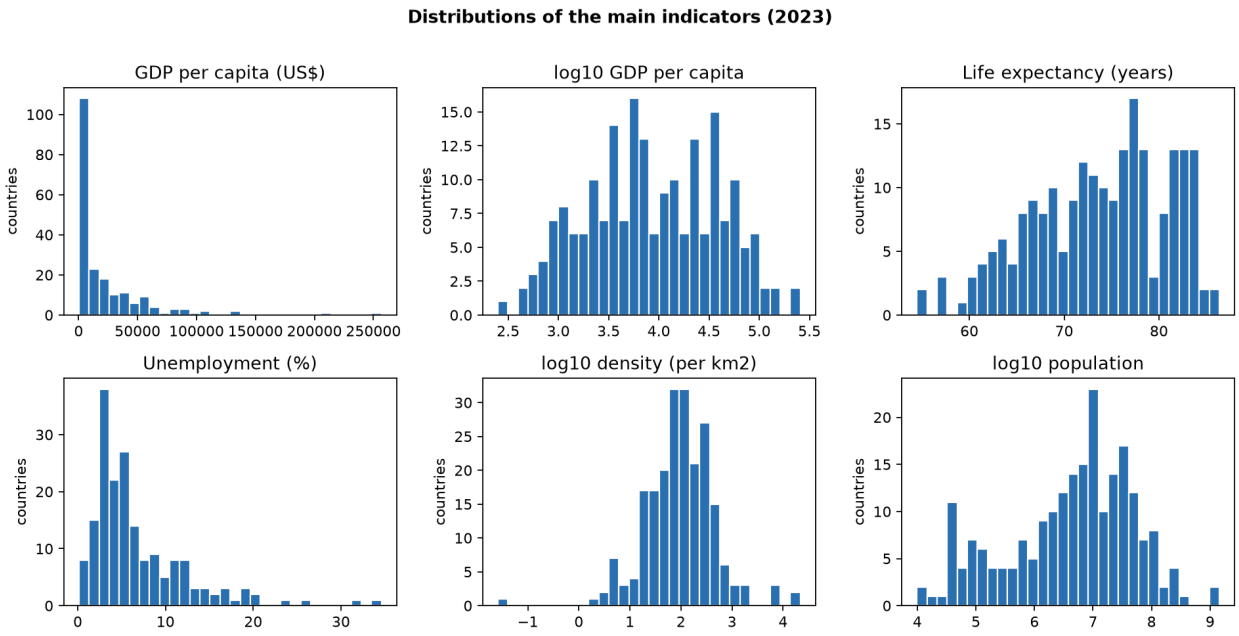


Figure 7. Indicator distributions.

Correlation analysis

Pearson measures linear association; Spearman uses ranks and captures monotonic relationships. Log transforms were used for GDP, population and density.

Relationship	n	Result
Raw GDP vs life expectancy	203	Pearson r = 0.62
Log GDP vs life expectancy	203	Pearson r = 0.85; Spearman rho = 0.86
Log GDP vs unemployment	175	Spearman rho = 0.04
Unemployment vs life expectancy	183	Spearman rho = -0.01
Log density vs life expectancy	216	Spearman rho = 0.24
Log population vs life expectancy	216	Spearman rho = -0.17

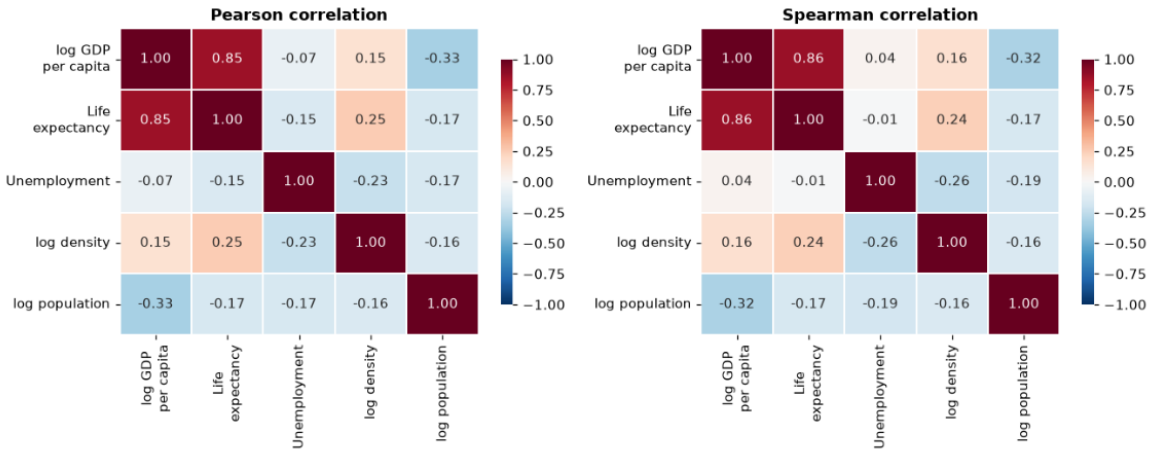
In [60]:

```
corr_cols = ["log_gdp", "life_expectancy", "unemployment", "log_density", "log_population"]
labels = ["log GDP\nper capita", "Life\nexpectancy", "Unemployment", "log density", "log population"]

print("Raw GDP vs life expectancy, Pearson r =", df[["gdp_per_capita", "life_expectancy"]].corr().iloc[0, 1].round(2))
print("log GDP vs life expectancy, Pearson r =", df[["log_gdp", "life_expectancy"]].corr().iloc[0, 1].round(2))

fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
for ax, method in zip(axes, ["pearson", "spearman"]):
    c = df[corr_cols].corr(method=method)
    sns.heatmap(c, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, center=0,
                xticklabels=labels, yticklabels=labels, linewidths=1, linecolor="white",
                cbar_kws={"shrink": 0.8}, ax=ax)
    ax.set_title(f"{method.capitalize()} correlation")
    ax.grid(False)
plt.tight_layout()
plt.savefig("figures/fig4_correlation_heatmap.png", bbox_inches="tight")
plt.show()
```

```
Raw GDP vs life expectancy, Pearson r = 0.62
log GDP vs life expectancy, Pearson r = 0.85
```



Original Jupyter cell 60: Pearson/Spearman heatmaps

In [59]:

```
d = df.dropna(subset=["gdp_per_capita"]).copy()
slope, intercept = np.polyfit(d["log_gdp"], d["life_expectancy"], 1)
xs = np.linspace(d["log_gdp"].min(), d["log_gdp"].max(), 100)
r = d["log_gdp"].corr(d["life_expectancy"])

fig, ax = plt.subplots(figsize=(9, 5.8))
ax.scatter(d["gdp_per_capita"], d["life_expectancy"], s=np.clip(d["population_wb"] / 8e6, 12, 400),
           color=BLUE, alpha=0.55, edgecolor="white", linewidth=0.6)
ax.plot(10 ** xs, slope * xs + intercept, color=ORANGE, lw=2, label=f"Linear fit on log GDP (r = {r:.2f}, n = {len(d)})")
ax.set_xscale("log")
for name in ["China", "India", "United States", "Nigeria", "Chad", "Monaco", "Japan", "Lebanon", "Equatorial Guinea"]:
    row = d[d["Country"] == name]
    if len(row):
        ax.annotate(name, (row["gdp_per_capita"].iloc[0], row["life_expectancy"].iloc[0]),
                    xytext=(5, -10), textcoords="offset points", fontsize=8, color="#333")
ax.set_xlabel("GDP per capita, 2023 (current US$, log scale)")
ax.set_ylabel("Life expectancy at birth (years)")
ax.set_title("Richer countries live longer — with diminishing returns")
ax.legend(loc="lower right", frameon=False)
ax.text(0.01, 0.01, "Bubble size = population", transform=ax.transAxes, fontsize=8, color=GREY)
plt.tight_layout()
plt.savefig("figures/fig1_gdp_vs_life_expectancy.png", bbox_inches="tight")
plt.show()
```



Original Jupyter cell 59: GDP and life expectancy



Figure 9. Pearson and Spearman heatmaps.

Main visual findings

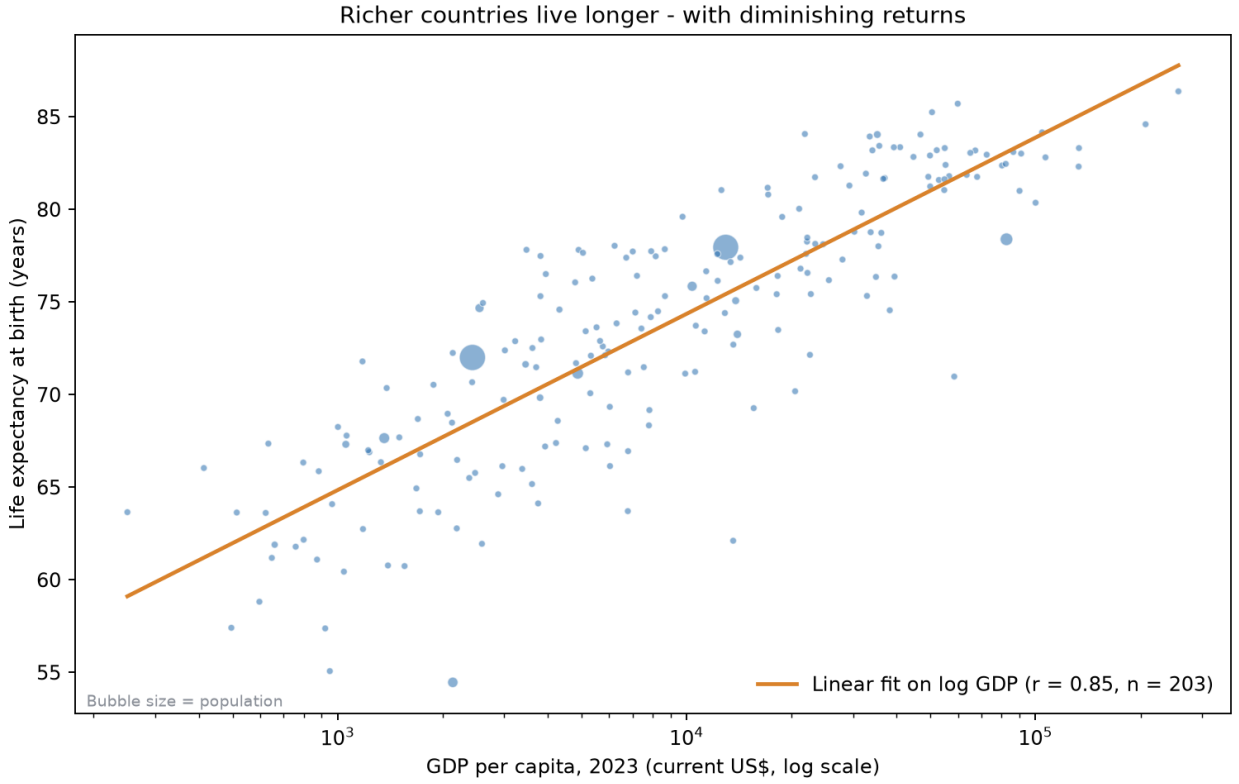


Figure 10. GDP and life expectancy.

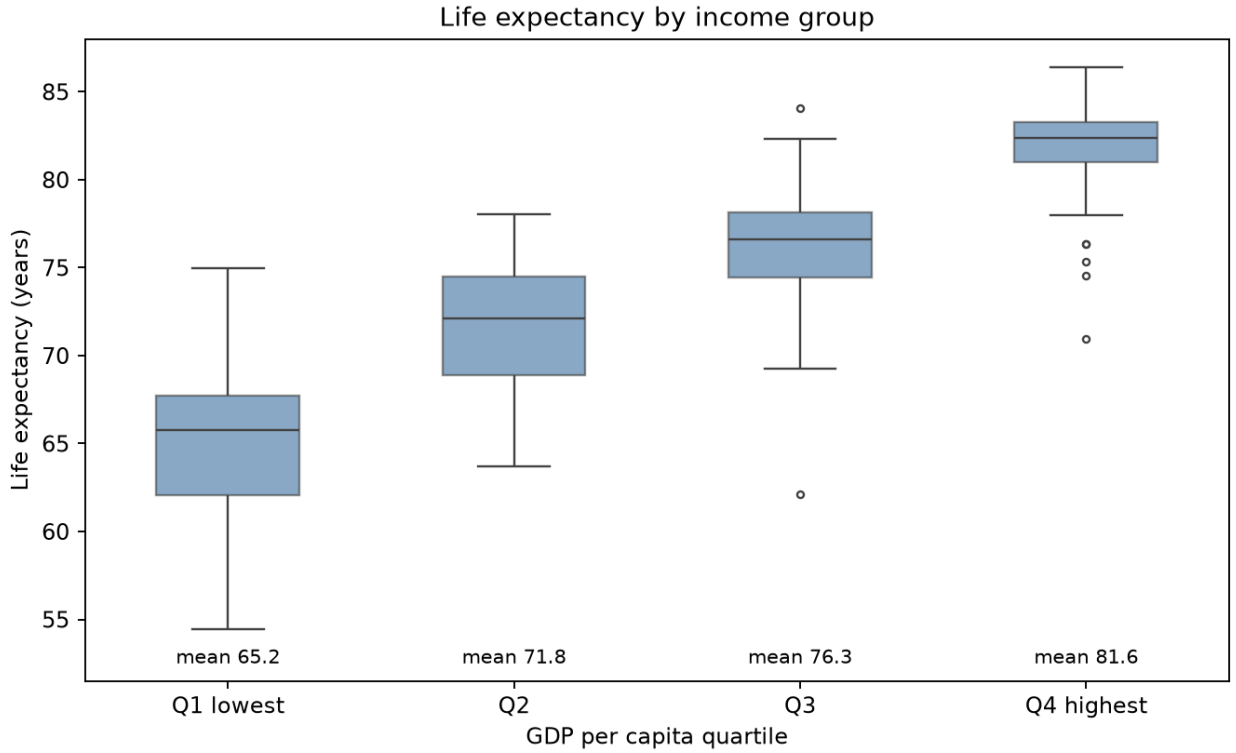


Figure 11. Life expectancy by GDP quartile.

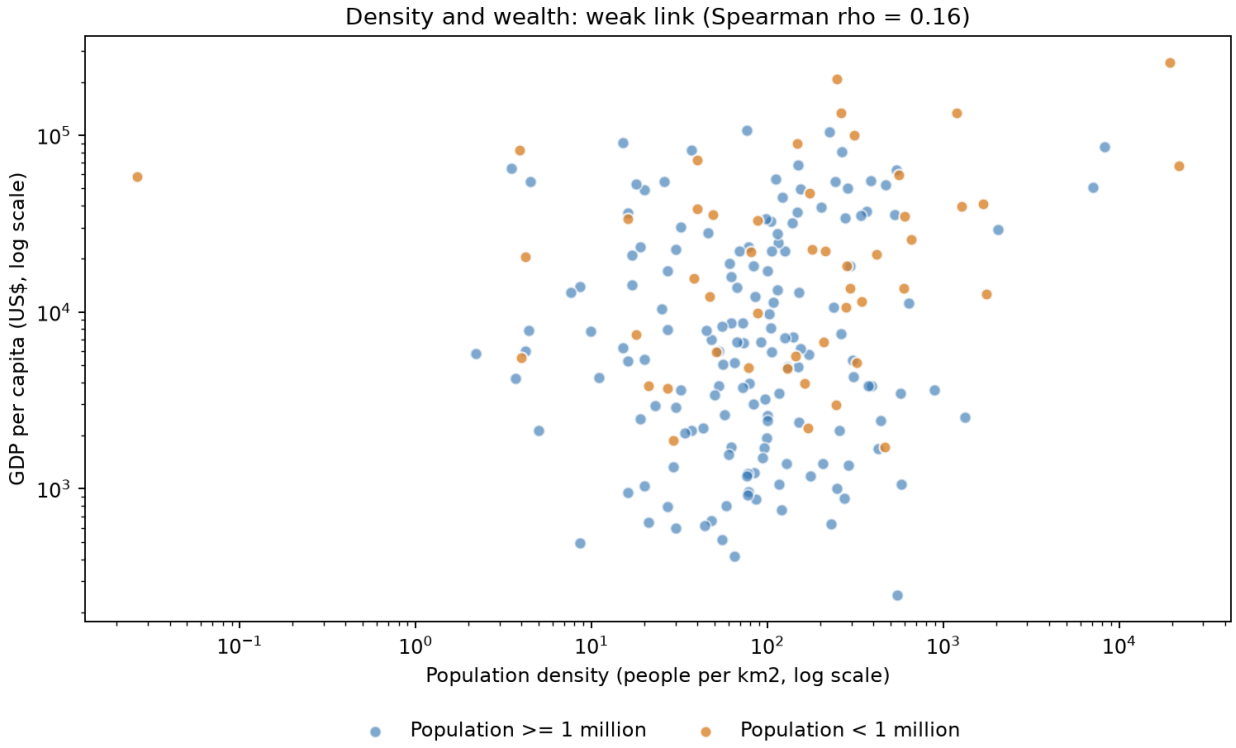


Figure 12. Density and GDP.

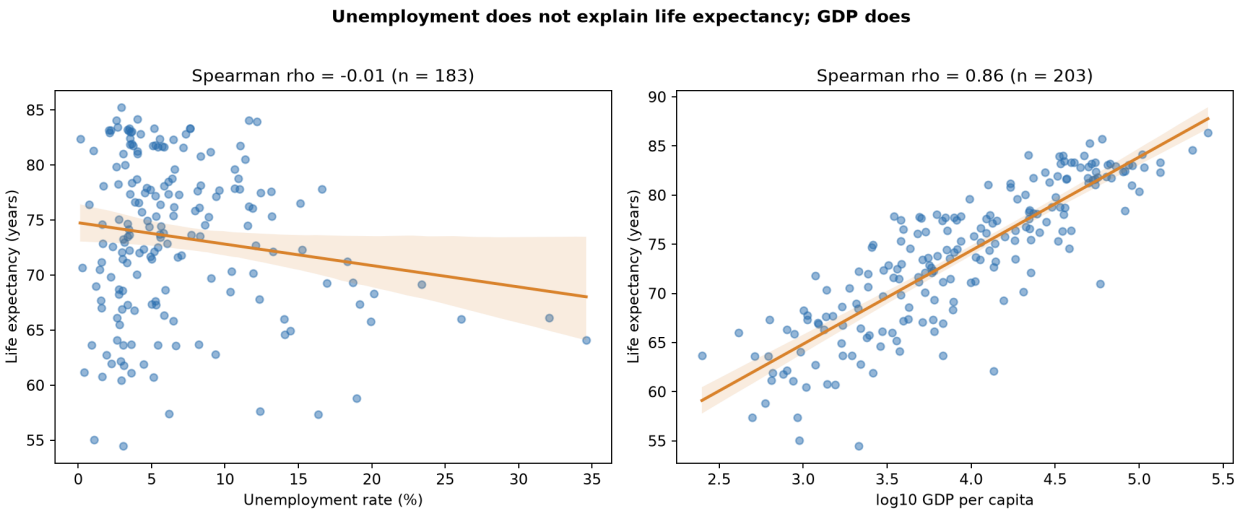


Figure 13. Unemployment comparison.

Optional machine-learning extension

The extension uses 175 complete rows, an 80/20 split and four features. The four-feature model reaches R2 = 0.736, MAE = 2.74 years and RMSE = 3.32 years. Log GDP alone reaches R2 = 0.711; five-fold cross-validation gives mean R2 = 0.74.

In [70]:

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

features = ["log_gdp", "unemployment", "log_density", "log_population"]
ml = df.dropna(subset=features + ["life_expectancy"])
X, y = ml[features], ml["life_expectancy"]
print("Rows used:", len(ml), "of", len(df))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

def evaluate(model, cols, name):
    model.fit(X_train[cols], y_train)
    pred = model.predict(X_test[cols])
    return {"model": name, "R2": round(r2_score(y_test, pred), 3),
            "MAE (years)": round(mean_absolute_error(y_test, pred), 2),
            "RMSE (years)": round(mean_squared_error(y_test, pred) ** 0.5, 2)}, pred

res_base, _ = evaluate(DummyRegressor(strategy="mean"), features, "Baseline (mean)")
res_one, _ = evaluate(LinearRegression(), ["log_gdp"], "Linear Regression: log GDP only")
res_all, pred = evaluate(LinearRegression(), features, "Linear Regression: 4 features")
display(pd.DataFrame([res_base, res_one, res_all]))

cv = cross_val_score(LinearRegression(), X, y, cv=5, scoring="r2")
print("5-fold CV R2:", cv.round(2), "| mean =", cv.mean().round(2))
print("Coefficients:", dict(zip(features, LinearRegression().fit(X_train, y_train).coef_.round(2))))
```

```
Rows used: 175 of 216
```

	model	R2	MAE (years)	RMSE (years)
0	Baseline (mean)	-0.019	5.53	6.51
1	Linear Regression: log GDP only	0.711	2.86	3.47
2	Linear Regression: 4 features	0.736	2.74	3.32

```
5-fold CV R2: [0.77 0.74 0.68 0.65 0.85] | mean = 0.74
Coefficients: {'log_gdp': np.float64(10.09), 'unemployment': np.float64(-0.08), 'log_density': np.float64(1.09), 'log_population': np.float64(0.3)}
```

Original Jupyter cell 70: regression metrics

Optional ML extension results

Model	R2	MAE	RMSE
Baseline mean	-0.019	5.53	6.51
Linear regression: log GDP	0.711	2.86	3.47
Linear regression: 4 features	0.736	2.74	3.32

Figure 15. Regression results.

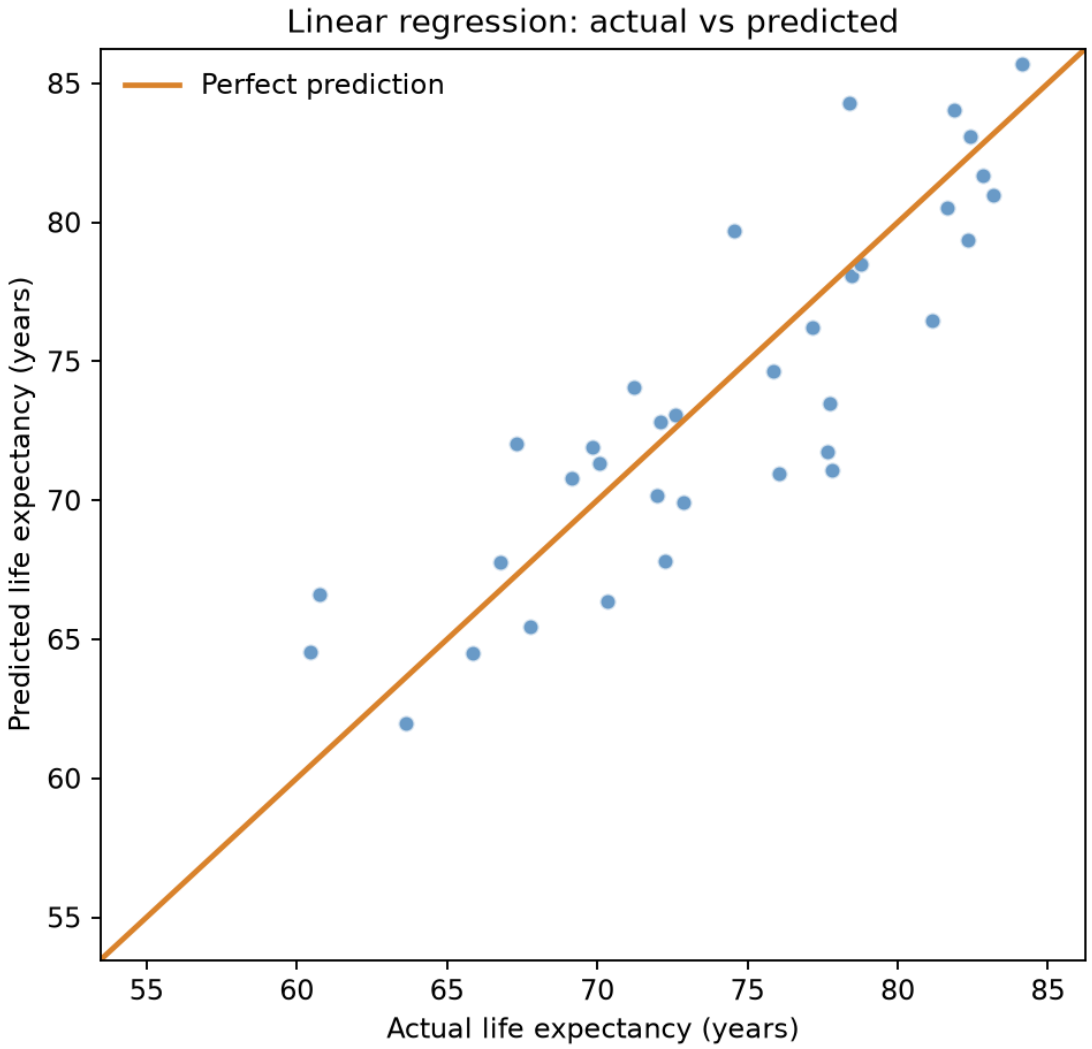


Figure 16. Actual versus predicted life expectancy.

Conclusions and limitations

Economic development has the strongest relationship with life expectancy. The poorest and richest GDP quartiles differ by about 16 years on average. Unemployment, density and population size are weak indicators alone. Limitations are the single year, country-level averages, mixed reference dates, current-USD GDP and the fact that correlation and prediction do not establish causality.

Sources

World Bank API: https://api.worldbank.org/v2/
World Bank documentation: https://datahelpdesk.worldbank.org/knowledgebase/articles/898581-api-basic-call-structures
Wikipedia: https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population_density
Repository: https://github.com/Broski228/Economic-Factors-and-Quality-of-Life-A-Global-Data-Analysis